SMART INDIA HACKATHON 2026

Problem Statement ID: SIH26156 | Category: Software | Theme: Miscellaneous

ULPF

Universal Log Pre-processing Framework

"Different Logs. One Standard. Trusted Everywhere."

PRODUCT REQUIREMENTS DOCUMENT

Product vision, users, scope, features and success metrics

Team & Role Allocation

Member	Role	Primary Ownership in This Document
Divyansh Rajat	Team Lead / System Architect	Overall architecture, scope, requirement baseline, traceability
Ashish Bhardwaj	Log Processing Engineer	Ingestion, format detection, template mining, parser factory
Sakshi	Normalization & Schema Engineer	Unified event model, OCSF/ECS mapping, schema registry, traceability
Parampreet Kaur	Backend & Integration Engineer	APIs, persistence, deployment, downstream connectors
Aryan Jeet	Frontend Engineer	Review console, dashboards, visualization requirements
Saksham	Testing, Documentation & Demo	Test strategy, benchmark methodology, acceptance evidence

Reference Documents

Ref.	Document	Source
R1	SIH 2026 Problem Statement SIH26156 — Universal Log Pre-processing Framework	Smart India Hackathon 2026 portal
R2	Drain3 — online log template mining library	logpai / IBM Drain3, open source
R3	OCSF — Open Cybersecurity Schema Framework	ocsf.io schema specification
R4	ECS — Elastic Common Schema field reference	Elastic documentation
R5	Splunk Common Information Model (CIM) documentation	Splunk Docs
R6	IBM QRadar DSM and DSM Editor documentation	IBM Documentation
R7	RFC 5424 — The Syslog Protocol	IETF
R8	ArcSight Common Event Format (CEF) and IBM LEEF specifications	Vendor specifications

1. Executive Summary

Every device in a modern enterprise writes down what it does, and almost none of them write it down the same way. A firewall emits Syslog, an application emits JSON, a legacy appliance emits a pipe-delimited string its vendor invented, and a cloud service emits a nested structure with entirely different field names. All of them may be describing the same login, the same blocked connection, the same policy violation. Before any SIEM, data lake or machine-learning pipeline can reason across them, somebody has to teach the platform what each source means.

That teaching step is manual, source-specific and unbounded. An organisation with a hundred log sources ends up maintaining something close to a hundred parsers, each of which breaks when a vendor changes a field. This is the bottleneck ULPF addresses.

ULPF — the Universal Log Pre-processing Framework — is an adaptive, lossless and traceable preprocessing layer that sits between heterogeneous log sources and whatever consumes them downstream. It ingests an event, determines whether its structure is already known, discovers the structure when it is not, extracts fields, maps them into a common taxonomy aligned with OCSF and ECS, and emits a standardised event — while retaining the original raw event and a provenance chain linking the two.

The product thesis in one sentence: *ULPF does not try to reinvent log parsing or replace a SIEM; it makes heterogeneous log preprocessing adaptive, lossless, traceable and vendor-independent, so that onboarding a new source becomes a repeatable operational procedure rather than a new development project.*

1.1 Product Snapshot

Attribute	Value
Product name	ULPF — Universal Log Pre-processing Framework
Problem statement	SIH26156 (Smart India Hackathon 2026, Software, Miscellaneous)
Product category	Data preprocessing / security data engineering middleware
Position in stack	Between log sources and SIEM / data lake / AI-ML pipelines — not a SIEM itself
Primary users	SOC analysts, detection engineers, platform and data engineers, compliance officers
Deployment model	Self-hosted, containerised (Docker Compose); air-gapped capable; no mandatory cloud dependency
Core differentiators	Adaptive onboarding · Lossless normalization · Bidirectional traceability · Schema evolution · Air-gapped by design
Target schemas	OCSF-aligned core with ECS-compatible mapping profiles and vendor extensions
MVP horizon	Working prototype demonstrating unknown-source onboarding end to end, with measured benchmarks

2. Problem Definition

2.1 Background

Enterprise and government networks generate log data from firewalls, routers, switches, servers, endpoint agents, databases, applications, IoT devices, VPN concentrators, identity providers and cloud control planes. This data is the raw material of security monitoring, incident investigation, compliance evidence and operational analytics.

The difficulty is not volume alone. It is heterogeneity: the same semantic fact is expressed in different formats, different structures, different field names and sometimes different vocabularies. A source IP address may appear as src_ip, sourceAddress, client_ip, originator or as the third positional field of an undocumented pipe-delimited record.

2.2 The Operational Bottleneck

The conventional response is a source-specific parser or content pack for each source. That approach works for a small, stable set of sources and degrades badly as diversity grows:

- **Linear growth of engineering effort.** Each new source requires a developer to study the format, write extraction logic, test it and deploy it.
- **Fragile to change.** A vendor firmware upgrade that adds or renames a field silently breaks downstream detection rules.
- **Silent information loss.** Fields that do not fit the target schema are frequently dropped during normalization, destroying evidence that may matter later in a forensic investigation.
- **No provenance.** Once an analyst sees a normalized value, there is often no reliable path back to the exact original record and the transformation that produced it.
- **Vendor lock-in.** Parsing effort invested in one platform's proprietary configuration format does not transfer to another platform.

Expressed as the arithmetic the team uses in the pitch: 100 log sources become 100 source-specific parsers, which become a maintenance burden that scales with the product of source count and format churn rate.

2.3 Problem Statement (as addressed)

Design and build a preprocessing framework capable of ingesting logs of arbitrary and previously unseen formats, discovering their structure with minimal source-specific development, normalizing them into a common extensible event model, preserving the complete original event and the provenance of every derived value, and delivering the result to downstream security and analytics platforms — including inside air-gapped environments.

2.4 Why Now

- Open, vendor-neutral event schemas (OCSF, ECS) have matured to the point where a common target taxonomy is realistic rather than aspirational.
- Online log template mining (Drain and its successors) is proven, streaming-capable and available as open-source components.
- Compact sentence-embedding models can now run entirely on-premise on CPU, which makes semantic field mapping feasible without sending log data to an external API — a hard requirement in defence and regulated sectors.
- Indian public-sector and critical-infrastructure deployments increasingly mandate on-premise or air-gapped operation, which excludes many SaaS-first log pipelines.

3. Product Vision, Positioning & Value

3.1 Vision Statement

A security team should be able to point ULPF at any log source — including one nobody has seen before — and get back consistent, standardised, analytics-ready events within minutes, without writing parser code, without losing a single byte of the original evidence, and without their data ever leaving their network.

3.2 Positioning

ULPF is deliberately not positioned as a SIEM, and not positioned as a replacement for Splunk, Elastic or QRadar. Those platforms already provide capable ingestion, parsing and normalization. ULPF is positioned as the vendor-agnostic preprocessing layer that sits in front of whichever of them an organisation already runs.

ULPF is…	ULPF is not…
A preprocessing and normalization framework	A SIEM or a detection engine
A producer of standardised, traceable events	A correlation, alerting or case-management product
A layer that feeds SIEMs, data lakes and ML pipelines	A storage or long-term retention platform
Assisted by AI for semantic ambiguity	An “AI understands any log automatically” black box
Vendor-independent and self-hosted	A managed cloud service

3.3 Value Proposition by Stakeholder

Stakeholder	Pain today	Value delivered by ULPF
SOC analyst	Inconsistent fields across sources make cross-source investigation slow and error-prone	One consistent event shape; one click back to the original raw record
Detection engineer	Every new source needs a parser before any rule can be written	Structure discovered automatically; a validated mapping is reusable thereafter
Platform / data engineer	Parser sprawl and format churn dominate maintenance effort	Mappings become versioned configuration instead of bespoke code
Compliance officer	Cannot show that a normalized value faithfully reflects the original record	Auditable provenance chain from raw to normalized event
Data scientist / ML engineer	Schema inconsistency dominates feature-engineering effort	Consistent structured events with stable field semantics
Air-gapped operator	Most modern pipelines assume cloud or Internet access	Fully offline operation with local models and local storage

3.4 The Five Product Principles

Every feature decision in this document is arbitrated against these five principles. They are the same five differentiators presented to the jury, restated as engineering constraints.

#	Principle	Engineering meaning
P1	Adaptive onboarding	An unseen source must be onboardable through discovery, mapping and validation — with zero or minimal source-specific code
P2	Lossless normalization	Normalization never destroys information; unmapped fields survive as extensions and the raw event is always retained
P3	Bidirectional traceability	Every normalized value carries provenance to its originating raw event and source field, and the raw event resolves forward to its normalized form

#	Principle	Engineering meaning
P4	Schema evolution	A stable versioned core schema plus dynamic namespaced extensions; adding a vendor field never breaks the core
P5	Air-gapped by design	No mandatory Internet, cloud API or external model call anywhere on the processing path

4. Target Users, Personas & Journeys

4.1 User Classes

User class	Technical level	Frequency of use	Primary interaction
Security analyst (Tier 1/2)	Medium	Daily	Consumes normalized events downstream; occasionally traces an event back to raw
Detection / content engineer	High	Weekly	Onboards sources, reviews proposed mappings, approves or corrects them
Platform administrator	High	Weekly	Configures ingestion endpoints, manages schema versions, monitors throughput
Compliance / audit officer	Low	Monthly / on demand	Retrieves provenance records and raw-event evidence for audit
Data / ML engineer	High	Continuous	Consumes normalized output via API, file export or message stream

4.2 Personas

Persona A — Meera, SOC Detection Engineer

Meera maintains detection content for a bank running two SIEMs after an acquisition. Roughly twice a month a new appliance appears on the network and she is asked how quickly she can “get it into the SIEM.” Today the honest answer is one to two days: study the format, write extraction logic, test it against sample data, get it deployed through change control. What she wants is to drop a sample file in, see a proposed field mapping with a confidence score, correct the two fields the system got wrong, approve it, and be done before lunch.

Persona B — Colonel Rao, Air-Gapped Network Administrator

Rao runs monitoring for an isolated network that has no Internet route by policy. Any product that phones home, downloads model weights at runtime or requires a cloud licence check is disqualified before evaluation. He needs a container bundle he can carry in on physical media, start with one command, and operate indefinitely offline.

Persona C — Anil, Compliance Officer

Anil has to demonstrate to an auditor that the events presented as evidence faithfully represent what the device actually recorded. He does not care about template mining. He cares that for any normalized field on screen, the system can show him the original record it came from, the mapping rule applied, and the schema version in force at the time.

4.3 Primary User Journey — Onboarding a New Source

Step	Today (without ULPF)	With ULPF
1	Receive sample logs from a new appliance	Receive sample logs from a new appliance
2	Manually inspect and document the format	Submit the sample to ULPF; format detection reports known or unknown
3	Write source-specific parser / regex / DSM	Template mining discovers the recurring structure automatically
4	Map fields to the platform's schema by hand	Semantic mapping proposes field mappings with per-field confidence
5	Test, iterate, debug edge cases	Engineer reviews proposals; corrects low-confidence fields in the console
6	Deploy through change control	Approve — the mapping is versioned and becomes reusable immediately
7	Maintain forever; re-fix on format change	Format drift is detected against the template; schema versioning absorbs the change

Step	Today (without ULPF)	With ULPF
Time	Typically 1–2 working days	Target: under 10 minutes for the prototype benchmark scenario

4.4 Secondary Journey — Forensic Trace-Back

An analyst investigating an incident sees a normalized event indicating a denied connection from 10.2.1.5. She selects the event and requests provenance. ULPF returns the trace identifier, the original raw event exactly as received, the source and receipt timestamp, the template that matched, the mapping version applied, and the per-field derivation showing that source.ip was extracted from positional field 3 of the vendor record. Nothing in that chain required the analyst to know anything about the vendor's format.

5. Goals & Success Metrics

5.1 Product Goals

ID	Goal	Rationale
G1	Reduce source-specific parser development to zero or minimal for new sources	This is the core bottleneck named in the problem statement
G2	Guarantee complete recoverability of the original event	Forensic and compliance value depends on it; it is also the claim most likely to be challenged
G3	Provide field-level provenance between raw, parsed and normalized representations	Makes normalization auditable rather than opaque
G4	Produce output aligned with an established open schema	Downstream consumption should not require learning a proprietary ULPF schema
G5	Absorb schema and format change without breaking existing data	Log sources change; a frozen schema is a dead schema
G6	Operate fully offline in a containerised deployment	Required for the defence, government and regulated deployments this PS targets
G7	Keep uncertain automated decisions under human control	Silent mis-mapping is worse than an unmapped field

5.2 Success Metrics

These are the seven metrics the prototype is validated against. Values in the target column are engineering targets and acceptance thresholds, not measured results; measured results replace them once benchmarking on the defined hardware and corpus is complete.

ID	Metric	Definition	Target
M1	New-source onboarding time	Elapsed time from first unseen log received to validated normalized events flowing	< 10 minutes
M2	Source-specific parser code	Lines of bespoke code required to onboard a new source	0 / minimal
M3	Field mapping accuracy	Correctly mapped fields ÷ total fields tested against ground truth	≥ 95%
M4	Raw-event recoverability	Original events byte-identically recoverable ÷ events ingested	100%
M5	Unknown-field preservation	Unmapped fields retained as extensions ÷ unknown fields received	100%
M6	Processing throughput	Events successfully normalized per second on defined hardware	Benchmarked
M7	Normalization latency	Time from ingestion to standardised event (p50 / p95)	Benchmarked

5.3 Anti-Metrics

Numbers the team explicitly refuses to claim, because they cannot be defended:

- “Supports every log format that exists.” Universality is measured as coverage across a defined test matrix of formats, structures, transports and schema conditions — not as an unbounded claim.
- “Processes billions of events per day.” The prototype measures real throughput on defined hardware; the architecture describes the horizontal path to scale. The two are reported separately.

- “AI understands any log automatically.” Automation is bounded by confidence thresholds with human validation for uncertain mappings.

6. Scope

6.1 In Scope — MVP / Prototype

Area	Included in MVP
Ingestion	File upload and directory watch, TCP and UDP Syslog listeners, HTTP/HTTPS POST endpoint, optional message-broker consumer
Formats	Syslog (RFC 3164 and 5424), JSON and NDJSON, XML, CSV/TSV, CEF, LEEF, key-value text, and arbitrary delimited or positional proprietary text
Detection	Deterministic format classification with confidence, plus fallback to structure discovery for unrecognised input
Discovery	Online template mining, variable-part extraction, candidate field typing
Mapping	Rule-based mapping for known field names plus embedding-based semantic mapping for unknown names, with composite confidence scoring
Normalization	Mapping into the ULPF unified event model with OCSF-aligned core fields and an ECS-compatible export profile
Preservation	Raw Event Vault storing the complete original event with an integrity digest
Traceability	Trace identifier linking raw, parsed and normalized representations, plus per-field derivation records
Schema	Versioned schema registry with namespaced vendor extensions
Human-in-the-loop	Review console for inspecting, correcting and approving proposed mappings
Output	REST query API, JSON and NDJSON export, Parquet export, and forwarding to a search index
Observability	Pipeline metrics, per-source statistics, confidence distribution, error and dead-letter handling
Deployment	Docker Compose bundle; offline installation; local embedding model

6.2 Out of Scope

- Detection rules, correlation logic, alerting and case management — these belong to the downstream SIEM.
- Long-term log retention and archival lifecycle management — ULPF preserves raw events for its retention window; archival is the data platform's concern.
- Log collection agents on endpoints. ULPF receives; it does not install collectors.
- Threat intelligence enrichment, geo-IP or asset enrichment. These are downstream enrichment concerns and are deliberately excluded from the preprocessing boundary.
- Multi-tenant SaaS operation, billing and tenant isolation.
- Encrypted or binary proprietary formats requiring vendor decryption libraries.

6.3 Future Scope (post-MVP)

Phase	Capability	Notes
V1.1	Automatic format-drift detection and re-learning	Alert when live events stop matching the approved template for a source
V1.2	Mapping package export / import	Share validated mappings between deployments — valuable for air-gapped sites
V1.3	Active-learning loop	Use analyst corrections to improve future mapping proposals for similar sources
V1.4	Additional schema profiles	CIM and CADF export profiles alongside OCSF and ECS

Phase	Capability	Notes
V2.0	Distributed multi-node deployment	Kubernetes operator, partitioned stream processing, HA vault

7. Feature Requirements

Features are grouped into epics. Priority uses MoSCoW: M = Must have for the prototype, S = Should have, C = Could have, W = Won't have this release. Each feature carries a forward reference to the functional requirements that specify it in the SRS.

7.1 Epic F1 — Universal Ingestion

ID	Feature	Description	Pri.	SRS ref.
F1.1	File ingestion	Upload or watch a directory for log files; handles rotation and partial reads	M	FR-1.1
F1.2	Syslog listeners	TCP and UDP listeners on configurable ports with framing handling	M	FR-1.2
F1.3	HTTP collector	Authenticated HTTP/HTTPS endpoint accepting single or batched events	M	FR-1.3
F1.4	Stream consumer	Consume from a message broker topic for high-volume sources	S	FR-1.4
F1.5	Source registry	Every ingestion channel is bound to a named, configured source identity	M	FR-1.5
F1.6	Backpressure & buffering	Queue and shed load gracefully rather than dropping events silently	M	FR-1.6

7.2 Epic F2 — Format Detection

ID	Feature	Description	Pri.	SRS ref.
F2.1	Deterministic classifier	Identify JSON, XML, CSV, CEF, LEEF, Syslog and key-value structures by grammar	M	FR-2.1
F2.2	Detection confidence	Emit a confidence value; low confidence routes to discovery rather than failing	M	FR-2.2
F2.3	Known-source fast path	Recognised source plus matching template bypasses discovery entirely	M	FR-2.3
F2.4	Encoding & framing handling	Character-set detection, multi-line event assembly, delimiter inference	S	FR-2.4

7.3 Epic F3 — Adaptive Parser Factory

ID	Feature	Description	Pri.	SRS ref.
F3.1	Template mining	Discover recurring log templates and variable positions online from a stream	M	FR-3.1
F3.2	Field extraction	Extract variable parts as candidate fields with inferred data types	M	FR-3.2
F3.3	Value-pattern typing	Classify values as IPv4/IPv6, port, timestamp, MAC, URL, hostname, user, action, etc.	M	FR-3.3
F3.4	Semantic mapping	Propose target schema fields for candidate fields using name and context similarity	M	FR-3.4
F3.5	Composite confidence	Score each proposed mapping from name, value, context and history signals	M	FR-3.5
F3.6	Mapping registry	Persist approved mappings so an onboarded source never re-learns	M	FR-3.6

ID	Feature	Description	Pri.	SRS ref.
F3.7	Parser generation	Emit a deterministic reusable parser artefact from an approved mapping	S	FR-3.7

7.4 Epic F4 — Normalization & Standardization

ID	Feature	Description	Pri.	SRS ref.
F4.1	Unified event model	A stable core event schema with defined field semantics and data types	M	FR-4.1
F4.2	OCSF alignment	Core fields aligned to OCSF categories, classes and objects	M	FR-4.2
F4.3	ECS export profile	Alternative rendering of the same event using ECS field names	S	FR-4.3
F4.4	Value normalization	Timestamps to UTC ISO-8601, actions to a controlled vocabulary, severity to a common scale	M	FR-4.4
F4.5	Extension namespace	Unmapped vendor fields preserved under a namespaced extensions object	M	FR-4.5
F4.6	Enrichment hooks	Defined extension points for downstream enrichment without altering the core	C	FR-4.6

7.5 Epic F5 — Lossless Preservation

ID	Feature	Description	Pri.	SRS ref.
F5.1	Raw Event Vault	Store the complete original event exactly as received, before any transformation	M	FR-5.1
F5.2	Integrity digest	Cryptographic hash per raw event to prove the stored copy is unaltered	M	FR-5.2
F5.3	Reception metadata	Record source, transport, receipt time and byte length alongside the raw event	M	FR-5.3
F5.4	Recoverability verification	A test harness proving byte-identical recovery for every ingested event	M	FR-5.4
F5.5	Retention policy	Configurable retention window and compaction for the vault	S	FR-5.5

7.6 Epic F6 — Traceability & Provenance

ID	Feature	Description	Pri.	SRS ref.
F6.1	Trace identifier	A single identifier binding raw, parsed and normalized representations of one event	M	FR-6.1
F6.2	Reverse lookup	Resolve a normalized event to its exact original raw record	M	FR-6.2
F6.3	Forward lookup	Resolve a raw record to the normalized event(s) derived from it	M	FR-6.3
F6.4	Field-level provenance	For each normalized field, record its source field, transformation and mapping version	M	FR-6.4
F6.5	Provenance API	Retrieve the full derivation chain for any event or field	M	FR-6.5

7.7 Epic F7 — Schema Registry & Evolution

ID	Feature	Description	Pri.	SRS ref.
F7.1	Versioned schemas	Every schema version is immutable and identified; events record the version applied	M	FR-7.1

ID	Feature	Description	Pri.	SRS ref.
F7.2	Backward compatibility rules	Additive-only changes to the core; removals require a new major version	M	FR-7.2
F7.3	Namespaced extensions	Vendor-specific fields live in isolated namespaces that cannot collide with the core	M	FR-7.3
F7.4	Mapping versioning	Mappings are versioned independently and pinned per source	M	FR-7.4
F7.5	Migration support	Reprocess historical raw events under a newer schema version from the vault	C	FR-7.5

7.8 Epic F8 — Human-in-the-Loop Review Console

ID	Feature	Description	Pri.	SRS ref.
F8.1	Onboarding wizard	Guided flow: submit sample → view discovered structure → review mapping → approve	M	FR-8.1
F8.2	Mapping review queue	Queue of proposals below the auto-accept confidence threshold	M	FR-8.2
F8.3	Side-by-side inspector	Raw event and normalized event displayed together with field linkage highlighted	M	FR-8.3
F8.4	Correction & override	Reassign a field to a different target and persist the correction	M	FR-8.4
F8.5	Approval audit trail	Record who approved which mapping, when, and what they changed	M	FR-8.5
F8.6	Pipeline dashboard	Throughput, per-source volume, confidence distribution, error rates	S	FR-8.6

7.9 Epic F9 — Downstream Delivery

ID	Feature	Description	Pri.	SRS ref.
F9.1	REST query API	Query normalized events with filtering, pagination and provenance expansion	M	FR-9.1
F9.2	File export	JSON, NDJSON and Parquet export for data-lake and ML consumption	M	FR-9.2
F9.3	Search index sink	Index normalized events into a local search engine for exploration	S	FR-9.3
F9.4	SIEM forwarding	Forward normalized events over Syslog, HTTP or broker to an external SIEM	S	FR-9.4
F9.5	Dead-letter output	Events that fail processing are captured with diagnostics, never silently dropped	M	FR-9.5

7.10 Epic F10 — Deployment & Operations

ID	Feature	Description	Pri.	SRS ref.
F10.1	Single-command deployment	docker compose up brings up the complete stack	M	FR-10.1
F10.2	Air-gapped installation	Offline image bundle with pre-baked model weights; no runtime downloads	M	FR-10.2

ID	Feature	Description	Pri.	SRS ref.
F10.3	Configuration as files	All sources, thresholds and profiles declared in version-controllable config	M	FR-10.3
F10.4	Health & metrics endpoints	Liveness, readiness and metrics exposed for operational monitoring	S	FR-10.4
F10.5	Role-based access	Separate viewer, approver and administrator roles	S	FR-10.5
F10.6	Horizontal worker scaling	Stateless processing workers scalable behind a partitioned queue	S	FR-10.6

8. User Stories & Acceptance Criteria

8.1 US-01 — Onboard an unknown source

As a detection engineer, I want to submit a sample from an appliance nobody has configured before and receive a proposed field mapping, so that I can start producing normalized events without writing parser code.

- **AC-1** Submitting a sample of an unrecognised format returns a discovered template within the interactive response budget.
- **AC-2** Each extracted candidate field is presented with an inferred type, a proposed target field and a confidence value.
- **AC-3** No source-specific code is written or deployed at any point in the flow.
- **AC-4** On approval, the mapping is persisted and subsequent events from that source are normalized on the fast path.

8.2 US-02 — Trust but verify a mapping

As a detection engineer, I want low-confidence mappings held for my review rather than applied silently, so that a wrong guess never becomes wrong data.

- **AC-1** Proposals scoring below the configured auto-accept threshold enter the review queue instead of being applied.
- **AC-2** The reviewer can see the evidence behind the proposal: field name, sample values, inferred type and similarity score.
- **AC-3** A correction overrides the proposal and is recorded with the reviewer's identity and timestamp.
- **AC-4** Events processed while a mapping is pending are still ingested, preserved and traceable; unmapped fields land in extensions.

8.3 US-03 — Recover the original event

As a compliance officer, I want to retrieve the exact original record behind any normalized event, so that I can present defensible evidence in an audit.

- **AC-1** Given a trace identifier, the API returns the raw event byte-identical to what was received.
- **AC-2** The response includes the integrity digest computed at ingestion and re-verified at retrieval.
- **AC-3** The response includes source identity, transport and receipt timestamp.
- **AC-4** Recoverability is demonstrated across the full test corpus, not sampled.

8.4 US-04 — Explain a normalized value

As an analyst, I want to know where a specific normalized field value came from, so that I can judge whether to trust it during an investigation.

- **AC-1** For any field in a normalized event, the API returns the originating source field or positional index.
- **AC-2** The response names the transformation applied and the mapping version used.
- **AC-3** The response includes the confidence score assigned at mapping time and whether it was human-approved.

8.5 US-05 — Nothing is lost

As a security engineer, I want vendor-specific fields that do not fit the common schema to be preserved, so that normalization does not destroy evidence.

- **AC-1** Every field present in the source event either maps to a core field or appears under the extensions namespace.
- **AC-2** The count of preserved fields equals the count of parsed fields for every test event.
- **AC-3** Extension fields are queryable through the same API as core fields.

8.6 US-06 — Run with no Internet

As an air-gapped network administrator, I want to install and operate ULPF on an isolated network, so that log data never leaves my perimeter.

- **AC-1** The offline bundle installs and starts with no outbound network access available.
- **AC-2** Semantic mapping operates using locally bundled model weights; no external inference call occurs.
- **AC-3** A network egress monitor observes zero outbound connections during a full processing run.

8.7 US-07 — Survive a format change

As a platform engineer, I want a vendor adding a new field to be absorbed without breaking existing pipelines, so that firmware upgrades are not incidents.

- **AC-1** An event containing a previously unseen field is processed successfully; the new field is preserved as an extension.
- **AC-2** Existing mapped fields continue to normalize unchanged.
- **AC-3** The divergence from the approved template is surfaced to operators rather than passing unnoticed.

9. Competitive Landscape & Differentiation

This section exists because the single most dangerous question the team will face is “Splunk, Elastic and QRadar already do this — what is new?” The honest answer begins by conceding that those platforms are genuinely capable, and then states the specific wedge ULPF occupies.

9.1 Existing Platforms

Platform	How it solves normalization	Where the source-specific effort remains
Splunk Enterprise Security	Common Information Model (CIM) with technology add-ons that parse and categorise known data sources	Onboarding commonly depends on identifying and configuring an add-on per data source; unusual sources need custom extraction
Elastic / Elastic Security	Ingest pipelines and processors normalizing into the Elastic Common Schema (ECS)	Arbitrary or unseen sources still require appropriate pipeline and processor configuration per log format
IBM QRadar	Device Support Modules (DSMs) identify, parse and normalize events; auto-detection for supported sources	Correct parsing depends on an appropriate DSM; unexpected formats may need a custom DSM or log-source extension built in the DSM Editor

9.2 Capability Comparison

The comparison below is written to be defensible. It does not claim these platforms cannot parse or normalize — they can, and do it well. It identifies where ULPF's design centre of gravity differs.

Capability	Established SIEM platforms	ULPF
Log ingestion	Mature, many transports	Mature transports, prototype scope
Parsing of known sources	Strong — large content libraries	Deterministic parsers, smaller library
Normalization	Yes — CIM / ECS / QRadar taxonomy	Yes — OCSF-aligned core with ECS profile
Unknown / undocumented source	Custom parser, DSM or pipeline configuration is typically required	Adaptive discovery and semantic mapping is the primary design centre
Raw-event handling	Raw events are collected and retained	Explicit Raw Event Vault with integrity digest is a first-class architectural component
Provenance	Event and source metadata	Field-level raw ↔ parsed ↔ normalized derivation is a core design goal
Product category	SIEM / analytics platform	Preprocessing layer feeding any of them
Vendor independence	Ecosystem-centric by design	Framework-level objective; output is not tied to one consumer
Air-gapped operation	Supported by several platforms	Assumed as the default deployment condition

9.3 The Defensible Claim

ULPF's novelty is not any single component. Template mining exists (Drain3). Open schemas exist (OCSF, ECS). Ingestion frameworks exist. The contribution is the orchestration layer that connects adaptive source discovery, semantic mapping, confidence-based validation, lossless preservation, field-level provenance, schema evolution and vendor-independent offline deployment into a single preprocessing workflow — and the evaluation framework that measures whether that workflow actually reduces onboarding effort without sacrificing evidence.

9.4 Coexistence Model

ULPF is designed to sit in front of existing platforms rather than displace them. A realistic deployment has ULPF receiving heterogeneous sources, normalizing and preserving them, and then fanning out to whichever combination of SIEM, data lake and ML pipeline the organisation already operates. Organisations running more than one analytics platform — a common outcome of mergers and of parallel IT/OT estates — gain the most, because preprocessing effort is spent once instead of once per platform.

10. Release Plan & Milestones

Milestone	Deliverable	Exit criteria
M0 — Foundation	Ingestion gateway, source registry, Raw Event Vault, trace identifier allocation	An event received on any transport is stored, digested and retrievable by trace ID
M1 — Known-format path	Deterministic detection and parsers for Syslog, JSON, XML, CSV, CEF, LEEF, key-value	Known-format events normalize end to end into the unified event model
M2 — Adaptive path	Template mining, field extraction, value typing, semantic mapping, confidence scoring	An unseen proprietary format produces a proposed mapping with per-field confidence
M3 — Human-in-the-loop	Review console, mapping registry, approval audit trail	A reviewer can correct and approve a proposal; the approved mapping is reused automatically
M4 — Traceability	Field-level provenance records and provenance API; reverse and forward lookup	Any normalized field resolves to its source field, transformation and mapping version
M5 — Delivery & deployment	Query API, exports, search index sink, Docker Compose bundle, offline install	Full stack starts offline with one command and serves normalized events downstream
M6 — Benchmark & evidence	Test corpus, ground-truth set, benchmark harness, measured results for M1–M7	Target values in §5.2 replaced by measured numbers with stated hardware and workload

10.1 Demonstration Scenario

The prototype demonstration is deliberately built around the single strongest moment: onboarding a source the system has never seen. The scripted sequence is — present an unseen vendor log; format detection reports it as unknown; template mining discovers the structure; fields are extracted and typed; semantic mapping proposes targets with confidence; a vendor-specific field that fits no core field is preserved as an extension; the normalized event is produced; and the trace identifier resolves back to the original raw record. Unknown source in, standardised traceable event out.

11. Assumptions, Dependencies & Constraints

11.1 Assumptions

- Log sources emit text-based or textually-representable events; encrypted or binary proprietary payloads requiring vendor libraries are out of scope.
- A sufficient sample of events from a new source is available for template mining to converge on stable templates.
- Reviewers with domain knowledge are available to adjudicate low-confidence mappings; ULPF reduces their workload but does not eliminate the role.
- The deployment host provides sufficient CPU for local embedding inference; GPU availability is not assumed.

11.2 Dependencies

Dependency	Type	Consequence if unavailable
Drain3 (template mining)	Open-source library	Adaptive discovery falls back to delimiter and grammar heuristics with reduced accuracy
Sentence-embedding model	Local model weights	Semantic mapping degrades to lexical and value-pattern matching only
OCSF / ECS specifications	External schema standards	Output schema would have to be proprietary, weakening downstream interoperability
Container runtime	Platform	Deployment story and air-gap packaging would require rework

11.3 Constraints

- No mandatory outbound network connectivity anywhere on the processing path — this constrains model choice, licensing and update mechanisms.
- The prototype is built and evaluated within the hackathon timeline by a six-member student team; scope is prioritised accordingly.
- Benchmarks must be reproducible on commodity hardware that the team can specify and the jury could verify.

12. Risk Register

ID	Risk	Impact	Likelihood	Mitigation
R1	An unknown format is incorrectly interpreted and produces wrong normalized values	High	Medium	Composite confidence scoring; auto-accept threshold; human review for uncertain mappings; raw event always preserved
R2	Vendor-specific fields do not fit the common schema	Medium	High	Namespaced extension fields plus retention of the original event — nothing is discarded
R3	Log formats change over time and break approved mappings	High	High	Versioned templates and schema registry; divergence detection surfaces drift to operators
R4	Event volume exceeds single-node processing capacity	Medium	Medium	Partitioned stream with stateless horizontally scalable workers; measured throughput reported honestly
R5	The semantic model proposes a plausible but wrong mapping	High	Medium	Deterministic rules take precedence where they apply; AI is an assistance layer, not the parser; low confidence never auto-applies
R6	Jury perceives the project as duplicating existing SIEM capability	High	Medium	Explicit coexistence positioning; comparison written to concede platform strengths and name the specific wedge

ID	Risk	Impact	Likelihood	Mitigation
R7	Unvalidated performance claims are challenged	Medium	High	Targets are labelled as targets; measured results reported with stated hardware and workload
R8	Scope expansion into detection and alerting dilutes the prototype	Medium	Medium	Detection, correlation and alerting are explicitly out of scope in §6.2

13. Glossary

Term	Definition
Log	A record describing an event that occurred in a system, device or application — what happened, when, where and often who caused it
Heterogeneous logs	Logs that differ in format, structure, field names, vocabulary and sometimes semantics even when describing similar events
Ingestion	Receiving log data into the system; the ingestion gateway is the controlled entry layer accepting file, network, API and streaming input
Parser	A component that breaks an unstructured or semi-structured log record into named fields
Template mining	Discovering recurring structural patterns across log messages and identifying which parts are constant and which are variable
Normalization	Converting differing representations of the same concept into a common structure and meaning
Standardization	Defining the common structure, field names, types and vocabulary that everyone converts into
Semantic mapping	Determining what a source field actually means and mapping it to the correct target field, rather than matching field names as strings
Confidence score	A composite measure of how certain the system is that a proposed field mapping is correct
Lossless normalization	Normalization that preserves all original information — unmapped fields survive as extensions and the raw event is retained
Provenance	The recorded history of a value: where it came from, what transformed it, and under which schema and mapping version
Bidirectional traceability	The ability to move from a normalized event to its raw source record and from a raw record forward to its normalized form
Extensible schema	A stable core schema plus namespaced extensions, allowing new vendor fields without breaking the core
Schema evolution	Safely updating the schema over time through immutable versions without invalidating existing data
Air-gapped deployment	Operation inside a network with no route to the public Internet and no mandatory cloud or external API dependency
OCSF	Open Cybersecurity Schema Framework — a vendor-agnostic, extensible schema for cybersecurity events
ECS	Elastic Common Schema — a common set of event fields enabling consistent search, analysis and correlation
SIEM	Security Information and Event Management — the platform that collects, correlates and analyses security events
Data lake	Centralised storage for large volumes of diverse data in raw and processed forms
Raw Event Vault	ULPF's immutable store holding the complete original event exactly as received
Trace ID	The identifier binding the raw, parsed and normalized representations of a single event
Dead letter	An event that could not be processed, captured with diagnostics rather than discarded